

Laboratory 2: Digital Input and Output with Buttons and LED

Learning Objectives:

Upon completing this laboratory, students will be able to:

1. Describe what digital inputs and outputs are, and give examples of each.
2. Identify and use the built-in buttons on the M5Stack development kit as input devices.
3. Connect an external LED to the M5Stack development kit correctly and safely.
4. Read button states using the M5Stack button functions and respond to user input.
5. Control an LED using Arduino functions such as `pinMode()` and `digitalWrite()`.
6. Use conditional statements (`if`) to make decisions based on button input.
7. Explain the difference between active-HIGH and active-LOW LED wiring configurations.

Key Concepts:

1. M5Stack Digital Input/Output (I/O) Pins

M5Stack provides GPIO (general purpose input/output) pins. See Illustration 2.1 for the pin mapping. However, each pin can be either input or output at a time (specifying by the `pinMode` function). There are three values that can be specified as a parameter for the `pinMode` function: **INPUT**, **OUTPUT**, **INPUT_PULLUP** (case sensitive). In this lab, you will only use the **OUTPUT** mode with the LED.



	GND	1	2	ADC1	GPIO35
	GND	3	4	ADC2	GPIO36
	GND	5	6	RESET	EN
GPIO23	MOSI	7	8	DAC0/AUDIO_L	GPIO25
GPIO19	MISO	9	10	DAC1/AUDIO_R	GPIO26
GPIO18	SCK	11	12	3.3V	
GPIO3	IO0/RXD1	13	14	IO1/TXD1	GPIO1
GPIO16	IO2/RXD2	15	16	IO3/TXD2	GPIO17
GPIO21	IO4/SDA	17	18	IO5/SCL	GPIO22
GPIO2	IO6	19	20	IO7	GPIO5
GPIO12	IO8/IIS_SCLK	21	22	IO9/IIS_WS	GPIO13
GPIO15	IO10/IIS_OUT	23	24	IO11/IIS_MCLK/BOOT	GPIO0
	HPWR	25	26	ADC0/IIS_IN	GPIO34
	HPWR	27	28	5V	
	HPWR	29	30	BATTERY	

Illustration 2.1: M5Stack Pins

2. Embedded Systems: Inputs and Outputs

An embedded system interacts with the physical world through inputs and outputs.

- Inputs provide information to the system.
- Outputs allow the system to respond or perform actions.

Examples:

Component	Type
Button	Input
Temperature Sensor	Input
LCD Display	Output
LED	Output
Speaker	Output

In this laboratory, the M5Stack buttons will be used as inputs, while the LCD display and external LEDs will be used as outputs.

The basic flow of an embedded system is:

Input → Processing → Output

For example:

Button Pressed → Program Detects Event → LED Turns ON

3. Digital Inputs and Digital Outputs

Many embedded devices operate using digital signals. A digital signal has only two possible states:

- HIGH
- LOW

These states can also be interpreted as:

Digital State	Meaning
HIGH	ON, True, 1
LOW	OFF, False, 0

Digital outputs can be used to control devices such as LEDs.

Digital inputs can be used to read the state of buttons and switches.

4. Reading Button Inputs

The M5Stack Core Basic contains three built-in buttons:

- Button A
- Button B
- Button C

The buttons can be accessed using:

M5.BtnA

M5.BtnB

M5.BtnC



Before reading button states, the program must update the button information:

M5.update();

This function should be called inside the `loop()` function.

Two commonly used button functions are:

- `isPressed()`

Returns **true** while the button is being **held down**.

```
if (M5.BtnA.isPressed()) {
  //Button A is currently pressed
}
```

- `wasPressed()`

Returns **true** only once when the button is first pressed.

```
if (M5.BtnA.wasPressed()) {
  //Button A has just been pressed
}
```

Example:

Press and hold the button							
Time:	----->						
Button:	Released	Pressed	-----				Released
isPressed:	false	true	true	true	true	true	false
wasPressed:	false	true	false	false	false	false	false

5. Making Decisions with if Statements

Programs often need to make decisions based on user input.

The `if` statement allows a program to execute code only when a condition is **true**.

Think of it like a gatekeeper: the code inside only runs if the condition passes.

Example:

```
if (M5.BtnA.wasPressed()) {
  M5.Display.println("Button A Pressed");
}
```

In this example, the message is displayed once only when Button A is pressed.

6. Controlling LEDs with Digital Outputs

An LED (Light Emitting Diode) is a simple electronic component that emits light when electrical current flows through it. The electrical current always flows in one direction, from anode to cathode. Illustration 2.2 shows the symbol of an LED.



Illustration 2.2: Electronic Symbol of LED

In Arduino, digital output pins can be used to control LEDs.

Before using a pin as an output, its mode must be configured:

```
pinMode(21, OUTPUT);
```

- To turn the LED ON: `digitalWrite(21, HIGH);`
- To turn the LED OFF: `digitalWrite(21, LOW);`

By combining button inputs and LED outputs, we can create simple interactive embedded applications.

Due to its characteristics, LED is an ideal choice for using as an output of a digital pin. Using LED as a digital output, we can easily observe the logic from the light. There are two ways to connect the LED to an output PIN, active LOW and active HIGH.

In particular, active LOW circuit will emit the light when the [output of P is LOW](#), and active HIGH circuit will emit the light when the [output of P is HIGH](#). To determine the correct logic, a programmer must understand the schematic of the circuits.

7. Connecting LED to the M5Stack Development Kit

Illustration 2.3 show the side/top-view locations of the digital I/O pins in the M5Stack kit.

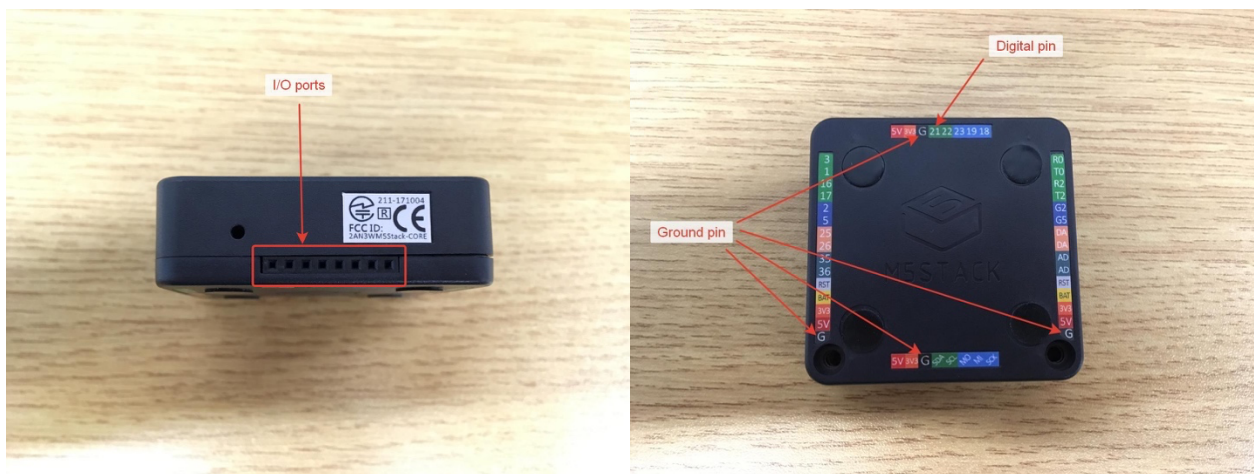


Illustration 2.3: Side/top-view locations of the digital I/O pins in the M5Stack development kit

In this lab, we will provide you with a small LED kit. The kits may come in variable forms. Illustration 2.4, show some examples of the kit with 1, 3 and 8 LEDs that are prewired with resistors. You

will be using the **digital PIN 21** to connect M5Stack to the LED in the kit. Depending on your specific LED kit, you will be using either the **GD (ground) PIN** or **3V3 (VCC) PIN** on the same side on M5Stack as the PIN 21 and choose the right configuration in Illustration 2.2 to properly make your LED kit works with M5Stack.

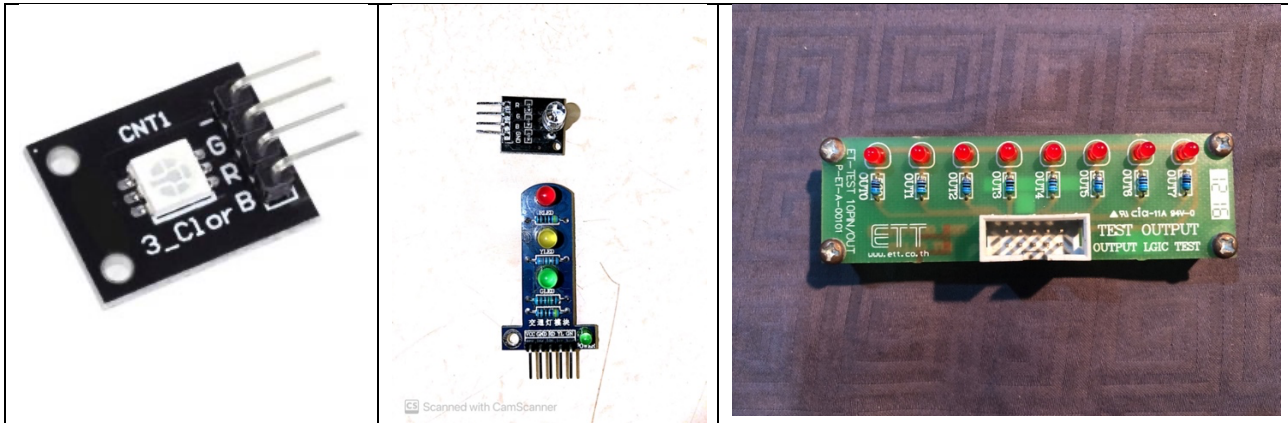


Illustration 2.4: Examples of LED Kits with prewired with resistors

Lab Exercises

Note: Save each task into different .ino, i.e., lab_2_1.ino, lab_2_2.ino.

Task 0: Connecting M5Stack to Computer

0.1 **Connect your M5Stack** to the computer using the USB cable, and open Arduino IDE.

0.2 **Verify board selection.** If you do not see **M5Core**, go to menu **Tools > Boards > M5Stack > M5Core**



0.3 **Verify serial port detection.** If Arduino IDE does not automatically detect M5Stack board, go to the menu **Tools > PORT** and select the right port for your set up.

Windows: COM3, COM4 (or COMx)

MacOS: cu.SLAB_USBtoUART, cu.usbserial-xxxx



0.4 If the board is not detected, try reconnecting the USB cable or restarting Arduino IDE. If the problem persists, ask a TA for assistance.

Task 1: Detecting Button Events

1.1 Detecting button events: `isPressed()`, `wasPressed()`.

Create a new Arduino sketch as follows and save as **lab_2_1.ino**.

```
#include <M5Unified.h>

void setup() {
  M5.begin();
  Serial.begin(115200);
  M5.Display.setTextSize(3);
  M5.Display.println("Lab 2 - Task 1");
  Serial.println("Lab 2 - Task 1");
}

void loop() {
  if (M5.BtnA.isPressed()) {
    Serial.println("Button A is pressed.");
  }
  if (M5.BtnB.wasPressed()) {
    Serial.println("Button B was pressed.");
  }
}
```

1.2 Upload and observe the output in the Serial Monitor when pressing Button A or B. You will see that the output in the Serial Monitor **is not displayed as expected**.

1.3 Identify the missing statement and add it to the sketch to make the correct message displayed in the Serial Monitor. **Observe the output in the Serial Monitor** when short press and long

press button A and B (see **Key Concepts** [4. Reading Button Inputs](#)).

1.4 Answer the questions in your lab sheet.

1.5 Modify the sketch to handle button pressed as following:

Button Single Pressed	LCD Output	Message to Serial Monitor
A	Hello	Button A
B	Welcome	Button B
C	Clear Screen	Button C

Note: Only one message should be displayed on the LCD at a time. (Hint: use `M5.Display.clear()` before printing a new message.)

Task 2: Controlling an LED and Understanding LED Configurations

2.1 Test the LEDs by connecting the LED's negative pin (cathode, marked '—') to the M5Stack's 'G' (Ground) pin. Then, connect the positive pin (anode) of the LED to the '3V3' pin on the M5Stack. Try each of the LED's color pins ('R' for Red, 'G' for Green, and 'B' for Blue) individually. The LED should light up if working. (At least two functional LED colors is required).

2.2 Connect the LED to the M5Stack development kit according to the **LED1** (active LOW) configuration in Illustration 2.5. (**Note: use pin 21 for P**)

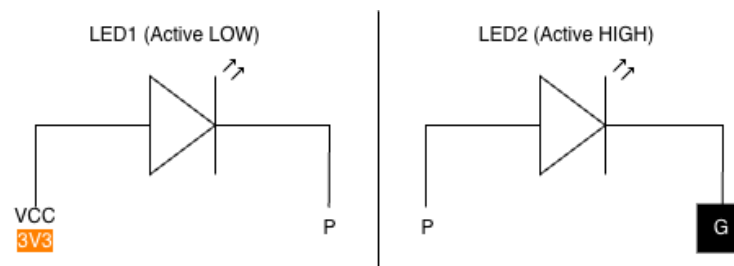


Illustration 2.5: Two methods for interfacing LED

2.3 Create a new Arduino sketch as follows and save as **lab_2_2.ino**.

```
#include <M5Unified.h>

const int ledPin = 21;
int ledState = LOW;

void setup(){
  M5.begin();
  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, ledState);
}

void loop() {
  M5.update(); // must call this every loop to read button states
  if (M5.BtnA.wasPressed()) {
    ledState = !ledState; // flip: LOW becomes HIGH, HIGH becomes LOW
    digitalWrite(ledPin, ledState);
  }
}
```

2.4 Prediction. Before uploading the sketch, predict whether the LED will be ON or OFF immediately after startup.

2.5 Upload the sketch and observe the LED when the program start and after each button pressed.

2.6 Change the LED connection to **LED2** in Illustration 2.2 and re-run the program by press **RESET** button (the **RED** button next to USB port). Compare the differences between LED1 and LED2. Which configuration is active high? Which configuration is active low? Then, answer the questions in your lab sheet.

Task 3: Controlling an LED with Multiple Buttons

Select an LED configuration (LED1 or LED2) and modify the code so that pressing any one of the buttons should toggle the LED exactly once. Initially, the LED must be OFF.

Hint:

Think carefully about which button function should be used.

Task 4: Controlling Multiple LED Colors with Multiple Buttons

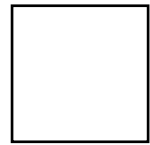
Connect two of the R, G, B pins to the M5Stack using active HIGH configuration (use pin **21**, **22** or **19**, **using other pins might conflict with internal circuit**). Use

- Button A to toggle one LED color,
- Button B to toggle another LED color, independently, and
- Button C to turn all LEDs **OFF**.

At start, LED must be **OFF**. Both LEDs may be **ON** simultaneously.

Task 5: Answer Questions

Task 6: Reflection assignment in myCourseVille.

**Lab 2: LED and M5Stack Buttons**

Section: _____ Date: _____

Members

Name: _____ Student ID: _____

Name: _____ Student ID: _____

Name: _____ Student ID: _____

Task		Graded by
1	Detecting Button Events • Button A – Hello • Button B – Welcome • Button C – Clear Screen	
2	Controlling an LED and Understanding LED Configurations • Press button A toggle LED state • Connect LED with LED1, LED2 configurations	
3	Controlling an LED with Multiple Buttons • LED OFF at start • LED toggle when any button is pressed	
4	Controlling an LED with Multiple Buttons • LED OFF at start • Button A – one color ON/OFF • Button B – another color ON/OFF • Button C – all colors OFF	

5. Answer the following questions.

5.1 What do you expect when you press and hold button A?

User Action	isPressed()	wasPressed()
Press and hold		
Continue holding		
Release		
Press again		

(----- More questions -----)

5.2 Based on your observations, explain the differences between `isPressed()` and `wasPressed()` in your own words.

5.3 Why in LED1 configuration does the LED turn on even though the output is LOW?

5.4 The program initializes `ledState` to LOW. Based on the LED behavior after startup, which configuration is active high, and active low? Explain your reasoning.

5.5 What is the role of `M5.update()` in the sketch? What will happen if you remove it?

6. Reflection assignment in myCourseVille.